

Love Messages App — Full Diagrams & Design (Firebase Architecture)

This version updates the system to use **Firebase** instead of a custom backend. All app functionality—authentication, database, storage, chat, and notifications—are handled via Firebase services.

Contents

1. High-level architecture (client / Firebase services)
 2. Component diagram (mobile features)
 3. User flows & screen map
 4. Sequence diagram: QR-based chat connection (via Firestore)
 5. Data model (Firebase collections)
 6. Firebase service mapping
 7. Media & sticker handling
 8. Security rules & privacy
 9. Deployment notes
 10. WhatsApp sticker packaging
-

1. High-level architecture

```
flowchart TD
    subgraph Mobile_App [Mobile App]
        A[Love Messages App]
        A --- M[Messages]
        A --- IQ[Image Quotes]
        A --- S[Stickers]
        A --- QC[QR Chat]
    end

    subgraph Firebase
        B1[Firebase Auth (Phone Sign-in)]
        B2[Cloud Firestore (Messages, Users, QR Pairs)]
        B3[Cloud Storage (Images, Stickers, Quotes)]
        B4[Firebase Cloud Messaging (Notifications)]
        B5[Firebase Functions (Optional Logic)]
        B6[Firebase Hosting / Dynamic Links]
    end

    A -->|SDK| B1
    A -->|Realtime sync| B2
    A -->|Upload/Download| B3
    A -->|Push| B4
    A -->|Trigger events| B5
```

All app logic resides client-side, with Firebase providing authentication, realtime database sync, and serverless triggers.

2. Component diagram

```
graph TD
    UI[Mobile UI]
    UI --> MessagesModule[Messages]
    UI --> QuotesModule[Image Quotes]
    UI --> StickersModule[Sticker Packs]
    UI --> QRChatModule[QR Chat]

    MessagesModule --> Firestore[(Firestore)]
    QuotesModule --> Storage[(Firebase Storage)]
    StickersModule --> Storage
    QRChatModule --> Firestore
    QRChatModule --> Functions[Cloud Functions]
```

3. User flows & screen map

Screens - Splash → Onboarding → Login (Phone auth) - Home (Tabs: Messages, Quotes, Stickers, Chat) - Love Messages: browse, favorite, share - Love Image Quotes: gallery, edit overlay text, share - Stickers: preview, add to WhatsApp - Love Chat: generate QR → scan → connect → chat

Flow summary: - QR code holds a short-lived pairing token written to Firestore. - The scanning user reads the token and updates pairing info, creating a shared chat document.

4. Sequence diagram: QR-based chat connection

```
sequenceDiagram
    participant A as User A (mobile)
    participant F as Firestore
    participant B as User B (mobile)

    A->>F: Create QR entry {userId, expiresAt}
    F-->>A: Returns docId (QR code content)
    A->>B: Shows QR (Firestore docId)
    B->>F: Reads QR doc, verifies not expired
    B->>F: Writes pair confirmation {userId}
    F-->>A: onSnapshot update triggers pairing confirmed
    A->>F: Creates chatRoom {A,B}
    B->>F: Syncs chatRoom messages via listener
    A<->>B: Realtime messaging (Firestore or RTDB)
```

5. Firebase data model (collections)

```
users/{uid}
- name
- phone
- avatarUrl
- qrCodeId

messages/{chatId}/items/{messageId}
- senderId
- contentType (text|image|sticker)
- content
- mediaUrl
- timestamp

chats/{chatId}
- members: [uidA, uidB]
- createdAt

quotes/{quoteId}
- imageUrl
- text
- tags

stickers/{packId}
- title
- coverUrl
- stickers: [{url, name}]

qrPairs/{qrId}
- userAId
- userBId
- expiresAt
```

6. Firebase service mapping

Feature	Firebase Service	Notes
Auth	Firebase Auth	Phone OTP or Google Sign-In
Messages & Chat	Firestore	Realtime message sync
Image Quotes	Firestore + Storage	Store metadata + image assets
Stickers	Storage	WebP files per pack
QR Chat	Firestore	Pair users via temporary QR docs

Feature	Firebase Service	Notes
Push Notifications	FCM	Notify on new messages
Cloud Functions	Functions	Optional for cleanup, expiry, or pairing validation

7. Media & sticker handling

- Use Firebase Storage for all images, quotes, and stickers.
- Cloud Functions can auto-generate thumbnails or WebP sticker conversions.
- Image uploads return public or signed URLs for display.

Workflow: 1. User selects an image / quote. 2. App uploads to Firebase Storage. 3. Firestore stores metadata link. 4. UI loads via `GlideAsyncImage(url)`.

8. Security & privacy

- Use Firebase Auth for identity.
- Define Firestore security rules:
- Only allow users to read/write their own data.
- Limit QR pair documents to owner or paired user.
- Deny expired QR codes.
- Enable Firebase App Check to prevent abuse.
- Encrypt chat content client-side if desired.

9. Deployment & infra

- No backend servers.
- All via Firebase Hosting, Firestore, Auth, Storage, Functions.
- CI/CD via Firebase CLI or GitHub Actions.
- Use Firebase Emulator Suite for local testing.

10. WhatsApp sticker packaging

- Store each sticker pack's WebP assets in `/stickers/{packId}`.
- Use Android Intent to add sticker pack to WhatsApp.
- For iOS: allow export as .zip for Shortcuts import.

Sticker requirements: - Max 512×512 px, WebP format. - Tray icon: 96×96 PNG. - Include metadata: name, author.

Summary

This Firebase-first design eliminates backend complexity: - All real-time chat, pairing, and media are handled via Firestore & Storage. - Authentication, hosting, and notifications are integrated. - Cloud Functions optionally enforce QR expiry and trigger cleanup.

This keeps the Love Messages app fully serverless, scalable, and easy to maintain.